2026 KDT FINAL PROJECT · 3조 백수 탈출기

바이브 코딩 팀 가이드

AI와 함께 개발할 때, 사람이 알아야 할 것들

대상: 팀원 6명 (AI 코딩이 처음인 사람 기준)
기준일: 2026-10-31 · 작성: 문서 담당
AI가 읽는 규칙은 저장소의 `AGENTS.md` 에 있습니다. 이 문서는 그 규칙의 **이유**와 **잘 쓰는 법**을 설명합니다.

목차

먼저 읽기 **한 장 요약** — 다음 쪽. 처음이라면 이 한 장만 읽고 시작해도 됩니다.

1. 이 폴더 사용법 — 무엇을 누가 읽나
2. 팀 원칙과 그 이유
3. 도구별 사용법 — Claude, ChatGPT·Codex, Gemini, Google AI Mode
4. 프롬프트 작성법 — 요청 템플릿, 좋은 요청과 나쁜 요청
5. 토큰·한도 아끼는 습관
6. 매일 쓰는 체크리스트와 숙련 단계
7. Git · PR · 작업 로그 흐름
8. 팀이 결정할 때 참고할 초안
9. 범위 외 아이디어
10. 부록 — WBS 일정 ↔ N-ID, 팀원 MD 통합 기록
11. 용어집 — 모르는 단어가 나오면 여기부터

> **이 문서를 읽는 순서**
> 한 장 요약 → 일하면서 막히는 장만 골라 읽기 → 모르는 단어는 맨 뒤 11장 용어집. 처음부터 끝까지 다 읽을 필요는 없습니다.

★ 한 장 요약

꼭 지킬 5가지

1. **CSS·디자인은 AI에게 맡기지 않는다.** 프론트 담당자의 영역.
2. **요청한 부분만 고치게 한다.** 다른 파일이 바뀌었으면 되돌린다.
3. **DB·API를 바꿨으면 로그를 남긴다.** `docs/logs/`
4. **정해지지 않은 건 AI가 정하지 않는다.** 선택지를 받아 팀에 가져온다.
5. **설명할 수 없는 코드는 커밋하지 않는다.**

도구별 시작 방법

- **Claude Code · Codex · Gemini 코딩 도구:** 이 폴더(`AGENTS.md`가 있는 곳)를 열면 규칙을 알아서 읽는다.
- **웹 채팅 (무료 포함):** 첫 메시지에 `docs/ai/PASTE.md` 블록을 붙여 넣는다.
- **Google AI Mode:** 코드보다 개념 이해·에러 검색용.

AI에게 요청할 때 — 이 틀을 복사해서 쓰기

```
[N-__] (기능 이름)
- 할 일:
- 범위: (파일 / 폴더 / 계층)
- 하지 말 것: CSS·스타일, 범위 밖 파일
- 확인 방법:
- 모르는 게 있으면 추측하지 말고 먼저 물어봐 줘. 끝나면 바꾼 파일과 이유를 짧게 정리해 줘.
```

시작 전

☐ 최신 `develop` 받기
☐ 오늘 기능의 N-ID 확인 (`features.md`)
☐ 최근 로그 훑어보기 (`docs/logs/`)

커밋 전

☐ 바뀐 파일 직접 확인 — CSS·범위 밖 변경 없음
☐ 실제로 실행해 봄
☐ 주석 1~2문장 + N-ID, 필요하면 로그

한도를 아끼려면

한 대화에는 한 작업만. 길어지면 "지금까지 요약해 줘" → 새 대화. 파일·에러는 관련 부분만 붙여 넣기.

막히면 어디를 보나

모르는 단어 → **11장 용어집** · 요청이 잘 안 먹힘 → 4장 · 한도가 빨리 참 → 5장 · Git·PR → 7장 · 왜 이런 규칙? → 2장

1 이 폴더 사용법

문서를 **AI가 읽는 것**, **사람이 읽는 것**, **둘 다 읽는 것**으로 나눴습니다.

```
(저장소 루트)
├── AGENTS.md     AI 공통 규칙 원본 — Codex 등이 자동으로 읽음
├── CLAUDE.md     AGENTS.md를 불러옴 + Claude Code 전용 몇 줄
├── GEMINI.md     AGENTS.md를 불러옴
├── README.md     사람용 첫 화면
├── .claude/settings.json   Claude Code 확인 창 설정 (CSS 수정, git push 등)
└── docs/
    ├── ai/       AI가 작업 전에 확인
    │   ├── features.md    기능 명세 사본 (N-ID 기준)
    │   ├── decisions.md   확정 · 미정 사항
    │   └── PASTE.md       웹 채팅용 붙여넣기 요약
    ├── logs/     사람·AI 공통 작업 로그 + 검수
    └── guide/    이 문서 (PDF와 원본 HTML)
```

파일	누가	언제
`AGENTS.md` 등 루트 파일	AI (자동)	코딩 도구가 대화를 시작할 때 스스로 읽습니다. 사람이 따로 할 일은 없습니다.
`docs/ai/features.md`	AI · 사람	기능을 만들기 전에 해당 N-ID의 입력·처리·출력을 확인합니다.
`docs/ai/decisions.md`	AI · 사람	확정된 것은 따르고, 미정(D-xx)인 것은 팀에서 정합니다. 바꾸는 사람은 문서 담당입니다.
`docs/ai/PASTE.md`	사람 → AI	저장소를 못 읽는 웹 채팅에 첫 메시지로 붙여 넣습니다 (3장).
`docs/logs/`	사람 · AI	DB·API·대량 백엔드 변경 후 작성하고, 지정된 검수 담당이 확인합니다 (7장).

md에 일부러 넣지 않은 것

DB 테이블·컬럼, API 형식, 폴더 구조, 환경 세팅, 학습 데이터 규칙은 **팀원이 직접 정할 영역**이라 AI 문서에서 뺐습니다. AI 문서에 초안을 적어 두면 AI가 그것을 "정해진 규칙"으로 받아들여 팀이 고민하기 전에 설계가 굳어집니다. 참고용 초안은 8장에 남겨 두었습니다.

"토큰 아끼기"를 md에 넣지 않은 이유

md는 모든 팀원의 AI가 매번 읽습니다. "짧게 답해라", "파일 읽지 마라" 같은 지시를 md에 넣으면 무료 사용자에게는 도움이 될 수 있어도, 자세한 설명이 필요한 순간이나 유료 사용자에게는 오히려 방해가 됩니다. 한도를 아끼는 건 **요청하는 방법**으로 하는 것이 맞습니다. 그래서 5장에 사람의 습관으로 정리했습니다.

2 팀 원칙과 그 이유

AI는 `AGENTS.md`의 규칙을 지키도록 되어 있지만, 사람이 이유를 알아야 AI가 규칙을 벗어날 때 알아챌 수 있습니다.

① 브랜치를 합칠 때 오류가 나지 않게

6명이 기능별로 각자 브랜치에서 개발합니다. 문제는 여러 기능이 함께 쓰는 파일입니다 — `pom.xml`, `package.json`, 공통 설정, 공통 응답 클래스, 라우터, Security 설정, DB 스크립트. AI는 요청하지 않아도 "김에" 이런 파일의 포맷을 정리하거나 이름을 바꾸는 경향이 있어서, 합칠 때 충돌이 납니다.

- 공유 파일을 고쳐야 하면 **Discord `#개발`에 먼저 알린다.**
- AI가 요청과 상관없는 파일까지 바꿨다면 그 부분은 되돌리고 커밋한다.
- 명세에 "계산을 공유한다"고 적힌 기능(N-16·N-20, N-17·N-18·N-21, N-21·N-22)은 담당자끼리 함수 하나를 함께 쓰도록 미리 얘기한다.

② 스타일·CSS는 프론트 담당자의 영역

AI는 "더 보기 좋게" 하려고 색·간격·클래스명을 쉽게 바꿉니다. 각자 AI에게 화면을 맡기면 화면마다 디자인이 달라지고, 프론트 담당자가 만든 테마(기본/블랙)가 깨집니다.

- AI는 CSS·`className`·인라인 스타일을 요청 없이 고치지 않고, 필요하면 **제안만** 합니다.
- Claude Code에서는 CSS 파일을 고치려 하면 확인 창이 뜹니다(`.claude/settings.json`). **내가 프론트 담당이 아니면 거절**하세요.

③ 주석 1~2문장 + N-ID

중간 시연과 최종 발표에서 각자 자기 코드를 설명해야 하고, 팀원끼리 서로의 코드를 읽어야 합니다. AI가 만든 코드는 길고 낯설기 쉬워서, 함수마다 "무엇을 하는지" 한 줄이 있으면 읽는 속도가 크게 달라집니다.

```
// N-12 나의 신용 리포트 — 기존·대안 점수를 함께 조회한다.
```

④ DB·API·대량 백엔드 변경은 반드시 로그

DB 컬럼 하나, API 응답 필드 하나가 바뀌면 다른 그룹의 화면과 기능이 바로 깨질 수 있습니다. 그래서 백엔드를 AI가 못 건드리게 막는 대신, **바꿨으면 반드시 기록하고 팀원이 확인하는** 방식으로 정했습니다.

로그를 반드시 남기는 경우
DB 관련 변경(테이블·컬럼·DDL) — 1건이라도
API 경로나 요청·응답 필드 추가·변경
한 작업에서 백엔드 파일 3개 이상 변경
빌드·의존성 파일이나 공통 설정 변경

⑤ 모르면 추측하지 않는다

등급 경계, 응답 형식, 타임아웃, 테이블 이름 등 아직 정하지 않은 것(`decisions.md`의 D-xx)이 많습니다. AI는 모르는 것도 그럴듯하게 지어내므로, 미정 항목은 상수로 빼고 `TODO(D-xx)`를 달도록 했습니다. AI가 선택지를 물어보면 **혼자 고르지 말고 팀에 가져오세요.**

> **미정 사항은 AI의 판단보다 사용자의 의견이 우선**
>
> AI가 "이게 더 좋아 보인다"며 미정 사항을 정해 버리면, 팀이 결정했을 때 다시 고쳐야 합니다. 그래서 `AGENTS.md`와 `decisions.md`에 "미정 항목은 AI가 스스로 정하지 않고, 선택지를 보여 주고 사용자가 고르게 한다"고 적어 두었습니다. AI가 이 규칙을 어기고 혼자 정했다면 그 부분을 되돌리고 다시 물어보게 하세요.

그 밖의 약속

- 합성 데이터만 쓰고, 실제 개인정보·API 키는 프롬프트에도 넣지 않는다.
- 대안 점수가 더 낮은 집단도 숨기지 않는다. 금리·한도는 "참고 정보"로만 보여 준다.
- AI는 `push`·병합을 스스로 하지 않는다. 올리는 건 사람이 한다.

3 도구별 사용법

도구마다 규칙 파일을 읽는 방식이 다릅니다. **저장소 폴더를 직접 여는 코딩 도구**는 규칙을 스스로 읽고, **웹 채팅**은 우리가 붙여 넣어야 합니다.

도구	규칙을 읽는 방식	할 일
Claude Code (VS Code 확장) Claude Pro 이상	루트 `CLAUDE.md` 자동 → `AGENTS.md` 불러옴	저장소 루트 폴더를 VS Code로 열고 시작하면 됩니다.
Codex ChatGPT Plus	루트 `AGENTS.md` 자동	저장소 루트에서 시작합니다.
Gemini 코딩 도구 Gemini CLI · Code Assist	루트 `GEMINI.md` 자동 → `AGENTS.md` 불러옴	저장소 루트에서 시작합니다. 불러오기가 안 되는 것 같으면 첫 메시지에 "AGENTS.md를 먼저 읽어 줘"라고 적습니다.
웹 채팅 Claude·ChatGPT·Gemini 웹, 무료 포함	저장소를 못 읽음	대화 첫 메시지에 `docs/ai/PASTE.md` 의 블록을 붙여 넣고, 고칠 파일 내용을 함께 붙입니다.
Google AI Mode	지침을 기억하지 못함	코드 작성보다 **개념 이해·에러 검색·학습**에 씁니다. 코드를 받아 쓸 때는 우리 규칙(CSS·주석)을 직접 확인합니다.

Claude Code를 쓸 때

- **계획 모드(plan 모드)**: 파일을 여러 개 고치는 작업은 먼저 계획을 보여 달라고 하세요. 계획을 읽고 "1, 3번만 진행해"처럼 고를 수 있습니다.
- **확인 창**: CSS 수정, `git push`, `rm` 등을 하려 하면 확인 창이 뜹니다. 무엇을 왜 하려는지 읽고 허락하세요. 습관적으로 "항상 허용"을 누르지 마세요.
- **파일 지정**: 대화에 파일을 지정(`@파일명`)하면 AI가 엉뚱한 파일을 찾아 헤매지 않습니다.
- Claude 앱(웹·데스크톱)은 문서 정리·설계 상의에, VS Code 확장은 실제 코드 작업에 나눠 쓰면 편합니다.

웹 채팅을 쓸 때

1. 새 대화 → `PASTE.md` 블록 붙여 넣기
2. 해당 기능의 `features.md` 내용(그 N-ID 부분만) 붙여 넣기
3. 고칠 파일의 **관련 부분만** 붙여 넣고 요청 (4장 템플릿)
4. 받은 코드를 내 파일에 옮긴 뒤, CSS·주석 규칙을 지켰는지 직접 확인

> **웹 채팅에서 자주 생기는 문제**
>
> 웹 채팅은 우리 저장소의 다른 파일을 모르기 때문에 이미 있는 함수와 같은 일을 하는 함수를 새로 만들거나, 다른 이름을 지어 오는 경우가 많습니다. 붙여 넣기 전에 "이미 비슷한 게 있는지"를 확인하세요.

심화: 스킬(Skills) — 익숙해진 뒤에

Claude Code에는 `.claude/skills/이름/SKILL.md`에 "특정 작업용 매뉴얼"을 넣어 두는 기능이 있습니다. 평소엔 이름만 알고 있다가 관련 작업에서 꺼내 읽습니다. 지금은 팀 설계가 정해지지 않아 넣지 않았습니다. 나중에 그룹별로 API 형식이나 모델 서버 구조가 확정되면, 그 그룹이 직접 자기 기능의 스킬을 만들어 보는 것을 권합니다. 꼭 지켜야 하는 규칙은 스킬이 아니라 `AGENTS.md`에 둡니다.

4 프롬프트 작성법

AI의 결과물이 나쁜 이유의 대부분은 **요청이 모호해서**입니다. 아래 네 가지만 적어도 결과가 확연히 달라지고, 다시 시키는 횟수가 줄어 한도도 아낄 수 있습니다.

요소	쓰는 법	예
무엇 (N-ID)	어떤 기능의 어떤 부분인지	N-12 리포트의 점수 조회 API
어디 (범위)	고칠 파일·계층을 명시	Controller와 Service만
하지 말 것	건드리면 안 되는 것	CSS, 다른 기능 파일, DB 스크립트
확인 방법	"됐다"의 기준	Swagger에서 200 응답과 두 점수 확인

요청 템플릿 (복사해서 쓰기)

```
[N-__] (기능 이름)
- 할 일:
- 범위: (파일 / 폴더 / 계층)
- 하지 말 것: CSS·스타일, 범위 밖 파일
- 확인 방법:
- 모르는 게 있으면 추측하지 말고 먼저 물어봐 줘.
- 끝나면 바꾼 파일과 이유를 짧게 정리해 줘.
```

좋은 요청과 나쁜 요청

× 모호함

```
신용 리포트 만들어줘
```

○ 범위가 분명함

```
[N-12] 나의 신용 리포트
- 할 일: 기존·대안 점수를 함께 돌려주는
  조회 API의 Controller·Service
- 범위: backend의 credit 관련 파일만
- 모델 서버는 아직 없으니 mock 응답으로
- 응답 형식은 미정(D-20)이니 먼저 제안해 줘
```

× 결과만 요구

```
에러 나 고쳐줘
(로그 300줄 붙여넣기)
```

○ 상황과 핵심만

```
회원가입 API 호출 시 500 에러.
- 한 일: N-02 가입 DTO에 연령대 필드 추가
- 에러 핵심 5줄: (Caused by 부분)
- 원인 후보를 먼저 설명하고,
  고칠 곳은 내가 확인한 뒤 진행해 줘
```

✕ AI에게 판단을 넘김

```
화면 좀 예쁘게 다듬어줘
```

◯ 로직만 맡김

```
[N-16] 슬라이더 값이 바뀌면 비율을
다시 계산하는 로직만 추가해 줘.
마크업·className·CSS는 그대로 두고,
스타일이 필요하면 제안만 해 줘.
```

더 잘 쓰는 요령

- **계획 먼저**: 큰 작업은 "바로 코드 짜지 말고 단계별 계획부터 보여 줘"로 시작합니다. 계획에서 틀린 걸 고치는 게 코드에서 고치는 것보다 훨씬 쌉니다.
- **작게 자르기**: "N-16 전체"보다 "N-16 비율 계산 함수 → 그다음 API → 그다음 화면 연결" 순서로.
- **설명 요청**: 받은 코드에서 모르는 부분은 "이 부분이 왜 필요한지 두 줄로 설명해 줘". 발표 준비가 따로 필요 없어집니다.
- **반대 의견 허락**: "더 단순한 방법이 있으면 말해 줘"를 붙이면 과하게 복잡한 코드가 줄어듭니다.
- **테스트로 목표 정하기**: "버그 고쳐 줘" 대신 "이 버그를 재현하는 테스트를 먼저 쓰고, 통과하게 고쳐 줘".

5 토큰·한도 아끼는 습관

무료 등급은 하루·시간당 한도가 빨리 찹니다. 한도는 **대화 길이**와 **붙여 넣는 양**에 가장 크게 좌우됩니다.

습관	이유와 방법
한 대화 = 한 작업	AI는 매번 대화 전체를 다시 읽습니다. 대화가 길수록 한 번의 질문이 비싸지고, 앞의 내용을 헷갈리기 시작합니다.
길어지면 요약해서 새 대화	"지금까지 결정한 것과 남은 일을 5줄로 요약해 줘" → 새 대화에 그 요약을 붙여 시작.
필요한 부분만 붙이기	파일 전체 대신 관련 함수만, 에러 로그 전체 대신 `Caused by` 주변 몇 줄만. CSV·모델 파일은 붙이지 않고 컬럼 목록 정도만.
출력 형태 지정	"바뀐 부분만 보여 줘", "설명은 3줄 이내" 처럼 **그때그때** 정합니다(md에 고정하지 않는 이유는 1장).
같은 걸 다시 시키지 않기	마음에 안 들면 처음부터 다시 시키기보다 "2번 함수에서 예외 처리만 바꿔 줘"처럼 고칠 곳을 짚습니다.
도구를 나눠 쓰기	개념 이해·검색은 Google AI Mode나 무료 채팅, 실제 코드 수정은 저장소를 읽는 도구. 설계처럼 어려운 질문에 좋은 모델을 아껴 씁니다.
같은 실수를 반복하면 끊기	AI가 같은 오류를 두세 번 반복하면 대화를 이어 가지 말고, 요약과 함께 새 대화를 엽니다.

> **한도를 가장 많이 아끼는 방법**
> 4장의 템플릿대로 처음부터 범위를 정해 주는 것입니다. "다시 해 줘"가 줄어드는 만큼 한도가 남습니다.

6 매일 쓰는 체크리스트와 숙련 단계

작업 시작 전

☐ 최신 `develop` 을 받았다 (`git pull origin develop`)
☐ 오늘 할 기능의 N-ID와 `features.md` 내용을 확인했다
☐ `docs/logs/` 에서 최근 로그를 훑어봤다 (내가 쓸 파일을 누가 바꿨는지)
☐ 관련된 미정 항목(D-xx)이 있는지 `decisions.md` 를 봤다

AI에게 맡길 때

☐ 요청에 N-ID · 범위 · 하지 말 것 · 확인 방법을 적었다
☐ 여러 파일이 바뀌는 작업은 계획부터 받았다
☐ AI가 물어본 미정 사항을 혼자 정하지 않고 팀에 가져갔다

끝나고 커밋 전

☐ 바뀐 파일 목록(diff)을 직접 확인했다 — 요청 밖 파일, CSS 변경이 없다
☐ 실제로 실행해 봤다 (빌드·화면·Swagger)
☐ 새 함수·API에 1~2문장 주석과 N-ID가 있다
☐ 이 코드를 팀원에게 **내 말로 설명할 수 있다**
☐ DB·API·백엔드 3개 이상·공통 설정을 바꿨다면 로그를 썼다

숙련 단계 — 하루하루 올라가기

1단계 · 따라 하기

- 템플릿을 그대로 복사해서 요청
- 함수 하나 단위로 작게 맡기기
- 받은 코드를 한 줄씩 설명 요청

2단계 · 범위 다루기

- 계획 먼저 받고 골라서 진행
- 대화가 길어지면 요약 후 새 대화
- diff 확인이 습관이 됨

3단계 · 팀과 맞추기

- 로그 작성·남의 로그 검수
- 공유 파일 변경 전 공지
- AI의 선택지를 팀 결정으로 연결

4단계 · 도구 다루기

- 도구를 목적별로 나눠 쓰기
- 테스트로 목표를 정하고 맡기기
- (선택) 그룹별 스킬 만들어 보기

7 Git · PR · 작업 로그 흐름

하루 작업 흐름

```
1. git checkout develop && git pull origin develop
2. git checkout -b feature/FP-12-credit-report     (Jira 이슈가 없으면 feature/기능명)
3. AI와 작업 → diff 확인 → 실행 확인
4. git commit -m "feat - N-12 신용 리포트 조회 API 추가"
5. (PR 직전) 최신 develop 다시 반영 → 다시 실행 확인
6. 해당되면 docs/logs/에 로그 파일 추가
7. git push → GitHub에서 PR 생성 → 검토 후 병합
```

구분	규칙
브랜치	`feature/…` , `fix/…` 를 `develop` 에서 만든다 → PR → `develop` → `main` . `main` · `develop` 에 직접 push 금지.
커밋 메시지	`행위 - 내용` , 관련 N-ID 포함. 행위: feat, fix, docs, style, refactor, test, chore. (형식 재논의 여지: D-17)
충돌	다른 사람의 코드를 지우는 쪽으로 해결하지 않는다. AI에게 "충돌 부분과 선택지를 보여줘"라고 하고, 필요하면 해당 담당자와 같이 본다.
비밀값	`.env` , `application-local.yml` 은 올리지 않는다. 실수로 올렸다면 바로 Discord에 알리고 키를 다시 발급한다.

PR 템플릿

```
## 작업 내용
-
## 관련 N-ID / Jira
- N-   / FP-
## 확인 방법
-
## 체크리스트
- [ ] 최신 develop 반영 후 실행·테스트 확인
- [ ] 공유 파일(빌드·공통 설정·공통 DTO·DB) 변경 시 사전 공지
- [ ] CSS 변경은 프론트 담당자가 요청·확인함
- [ ] 로그 조건에 해당하면 docs/logs/ 로그 추가
- [ ] 비밀키·개인 설정 파일 미포함
```

작업 로그와 검수

- 작업 하나에 파일 하나: `docs/logs/YYYY-MM-DD_작성자_요약.md` — 한 파일에 이어 쓰면 합칠 때 충돌이 납니다.
- 양식은 `_TEMPLATE.md` 를 복사합니다. AI에게 "이번 작업 로그를 템플릿대로 써 줘"라고 하면 초안을 만들어 줍니다. 단, **"확인함"에는 실제로 해 본 것만** 남기세요.

- 검수 담당과 PR 병합 절차는 `docs/logs/README.md`에 정리되어 있습니다. 다른 사람의 로그를 읽다가 문제가 보이면 Discord `#개발`에 알려 주세요.
- AI는 검수 칸에 대신 서명하지 않습니다. 검수는 사람이 합니다.

8 팀이 결정할 때 참고할 초안

팀원 MD에 들어 있던 설계 초안입니다. **정해진 것이 아니며 AI 문서에서는 뺐습니다.** 팀에서 D-xx를 결정할 때 출발점으로 쓰고, 결정되면 문서 담당이 `decisions.md` 확정 표에 올립니다.

ID	항목	팀원 문서의 초안
D-20	공통 응답 형식	안 A: `{ success, data, error{code, message} }` · 안 B: `{ success, data, message, errorCode }`
D-18	외부 호출 타임아웃	안 A: FastAPI 연결 1초 / 응답 3초 (배치 30초) · 안 B: 연결 3초 / 읽기 5초. 공통 의견: Gemini가 실패해도 점수 등 핵심 기능은 멈추지 않고 요약만 생략
D-14	저장소 폴더 구조	`frontend/` `backend/` (Maven) `model-server/` (FastAPI) `ml/` (학습) `db/` `docs/` . 모델 서버 폴더명은 `model-server` / `ml-server` 로 갈렸음
D-17	커밋 형식	현재 `행위 - 내용` · 대안 `타입: 내용` (Conventional Commits)
D-01	등급 경계·점수 변환	부실 확률 → 300~900점 변환(PDO: odds 20:1 = 600점, odds 2배마다 +50점). 명세 초안 경계: Good 780↑ · Standard 665~779 · Poor 664↓
D-15	프론트 도구	JavaScript 확정(TypeScript 미사용). 차트는 문서 담당의 UI 시안(Claude 아티팩트)에서 쓴 방식을 따를 예정 — 라이브러리 이름 확인 필요. 그 밖의 후보: Vite, React Router, Axios
—	DB 명명	대문자 스네이크, 모든 테이블·컬럼에 `COMMENT` , 금액 `NUMBER(15)` , 한글은 `VARCHAR2(n CHAR)`
—	모델 서버 운영	모델은 서버 시작 시 1회 로드, 모델 서버는 DB 접근 없이 계산만, 백엔드와 내부 키(`X-Internal-Key`)로 통신
—	개발 환경	JDK 17, Node 20 LTS, Python 3.11, STS 4 + Lombok, STS 인코딩 UTF-8 통일

9 범위 외 아이디어

팀원 MD에는 있었지만 **기능 명세 v1에 N-ID가 없는** 내용입니다. 명세에 없는 기능은 AI가 만들지 않도록 했으므로, 하고 싶다면 먼저 팀에서 명세에 추가할지 정해야 합니다.

아이디어	내용	가까운 기능
관리자 PD 일괄 재계산	전체 합성 사용자의 부실 확률을 관리자가 한 번에 다시 계산	N-25, N-26
관리자 리스크 히트맵	행정동·업종별 평균 부실 확률을 카카오맵 히트맵과 표로 표시	N-24, N-26
고위험군 목록	부실 확률이 높은 사용자 목록 조회	N-26
지도 기반 현금흐름 시뮬레이터	지도에서 행정동·업종 선택 → 상권 매출을 반영해 월별 현금흐름 계산	N-21, N-24
스트레스 테스트 AI 요약	금리·소득 시나리오 결과를 Gemini로 쉬운 말 요약	N-17, N-31
사회초년생 별도 직군	`NEW_GRAD` 를 직군으로 추가하는 안 — 직군은 3분류로 확정됐으므로, 연령대·금융 이력 길이로 표현하는 쪽이 맞음	N-08, N-13
모델 서버 장애 시 간이 점수	규칙 기반 대체 점수를 보여 주는 안 — "점수는 항상 모델이 계산" 의견과 충돌해 보류	N-12

10 부록

A. WBS 일정 ↔ N-ID

출처: WBS 자동화 시트지 「WBS (작업공간)」 탭, 2026-10-30 기준. 일정은 WBS 원본이 우선합니다.

3단계 · 1차 설계 및 개발 (10/26 ~ 11/18)

작업	파트	기간	관련 N-ID
시스템 구조 및 API 규격 확정	(미정)	10/26 ~ 10/30	01~10, 12~27, 30~32
데이터 수집, 전처리 및 피처 구성	데이터 모형	10/26 ~ 11/04	05, 12~18, 20~26
DB 및 공통 API 기반 구축	백엔드	10/29 ~ 11/04	1~5, 12, 14, 16, 25~27
공통 레이아웃 및 화면 구조	프론트엔드	11/02 ~ 11/05	1, 8, 12
기존, 대안 신용평가 모델 개발	데이터 모형	11/02 ~ 11/10	12, 14, 18, 25
회원가입, 로그인 인증 API	백엔드	11/02 ~ 11/10	1, 2
데이터 입력, 모의 연동 API	백엔드	11/04 ~ 11/09	5
회원가입, 로그인, 데이터 입력화면	프론트엔드	11/04 ~ 11/10	1, 2, 5
SHAP 영향 요인 분석	데이터 모형	11/09 ~ 11/13	14, 23
신용평가 예측, 리포트 API	백엔드	11/09 ~ 11/13	12, 14
신용 리포트 및 영향 요인 시각화	프론트엔드	11/09 ~ 11/13	12, 14
마이 페이지, 계정 복구 API	백엔드	11/09 ~ 11/13	3, 4
직군별 화면 분기 및 공통 UI 구현	프론트엔드	11/09 ~ 11/13	8
소득 변동성 진단 기본 기능 개발	데이터 모형	11/09 ~ 11/13	22
관리자 모델 성능 비교, 통계 기본 구현	백엔드	11/09 ~ 11/13	25, 26
마이페이지, 자산부채 건전성 화면	프론트엔드	11/10 ~ 11/13	4, 16
관리자 계정, 권한 관리 기본 구현	백엔드	11/10 ~ 11/13	27
회원 인증, 계정 관리, 신용평가 API 단위 테스트	백엔드	11/11 ~ 11/13	1~5, 12, 14, 27
핵심 서비스, 관리자 화면 기능 테스트	프론트엔드	11/12 ~ 11/16	1~5, 8, 12, 14, 16, 25~27
핵심 서비스 연동 및 중간 시연 점검	팀원 전체	11/16 ~ 11/18	1~5, 8, 12, 14, 16, 22, 25~27

4단계 · 2차 개발 (11/19 ~ 12/03)

작업	파트	기간	관련 N-ID
직군별 신용 분석 및 비교 통계 개발	데이터 모형	11/19 ~ 11/25	13, 15, 26
DSR, 시나리오 시뮬레이션, 재무 진단 고도화 API	백엔드	11/19 ~ 11/25	16, 17, 20
직군별 특화 분석 및 비교 화면	프론트엔드	11/19 ~ 11/26	13, 15, 21~24
신용, 금리, 현금흐름 시뮬레이터 화면	프론트엔드	11/19 ~ 11/27	17, 18, 21
현금흐름, 상권 분석 고도화	데이터 모형	11/19 ~ 11/27	21, 22, 24
직군별 시뮬레이션, 분석 API	백엔드	11/19 ~ 11/27	18, 21~24
마이페이지, 관리자 기능 고도화 API	백엔드	11/23 ~ 11/27	4, 25~27
게시판 기능 개발	백엔드	11/24 ~ 11/27	6
마이페이지, 관리자 대시보드 고도화	프론트엔드	11/23 ~ 11/30	4, 25~27
화면 환경 설정 기능 개발	프론트엔드	11/24 ~ 11/30	9
금융 용어 챗봇, AI 요약 API	백엔드	11/23 ~ 12/01	31, 32
신용평가 모델 성능 개선 및 검증	데이터 모형	11/24 ~ 12/02	12, 14, 25
AI 요약, 금융 용어 챗봇, 쉬운 금융사전 UI	프론트엔드	11/25 ~ 12/02	30~32
직군별 분석 및 모델 고도화 결과 검증	데이터 모형	11/27 ~ 12/02	12~18, 20~26
추가 기능, AI 서비스 API 테스트	백엔드	11/30 ~ 12/02	4, 6, 13, 15~18, 20~27, 31, 32
추가 기능, 시뮬레이터, AI 화면 테스트	프론트엔드	11/30 ~ 12/03	4, 6, 9, 13, 15~18, 20~27, 30~32
추가 기능 연동 및 개발 완료 점검	팀원 전체	12/02 ~ 12/03	6, 9, 13, 15, 17, 18, 20~24, 30~32

B. 팀원 MD 통합 기록 (2026-10-30)

팀원·조장이 각자 만든 AI 바이브 코딩용 MD 4묶음을 하나로 합친 기록입니다. 출처 표기: GH = AGENTS - GH.md · AV = Avenue MD · JWC = JWC MD · M3 = MD 3. 원본 폴더는 수정하지 않았습니다.

원본 → 통합본 대응

원본	통합본에서의 위치
`AGENTS - GH.md`	`AGENTS.md` 4-3(모델 무결성), 5-2·5-3·5-5·5-6 / 일부 `RECOMMENDED.md` 5장
`JWC MD/AGENTS.md`	`AGENTS.md` 뼈대(개요·문서 지도·시스템 구성·구현 순서·모를 때)
`JWC MD/CLAUDE.md`	`CLAUDE.md` (`@AGENTS.md` 불러오기 방식 유지)
`JWC MD/docs/spec/features.md`	`docs/spec/features.md` (**내용 변경 없음** — 원본 시트 사본이므로)
`JWC MD/docs/decisions.md`	`docs/decisions.md` (10-30 팀 결정 반영, D-17~D-20 추가)

원본	통합본에서의 위치
`Avenue MD/README.md`	`README.md` (기능 표를 N-ID 기준으로, 구조·문서 목록 갱신)
`Avenue MD/CLAUDE.md` 1부	`AGENTS.md` 1·3·5장, `README.md`
`Avenue MD/CLAUDE.md` 2부(Karpathy 원칙)	겹치는 부분 → `AGENTS.md` 5-2 / 나머지 → `RECOMMENDED.md` 1·2장
`Avenue MD/docs/*.md` 7개	`docs/` 같은 이름으로 유지 + 정합화(아래 4장)
`MD 3/CLAUDE.md`	`AGENTS.md` 5-1, `CLAUDE.md` (Edit 도구·토큰 절약), `CONVENTIONS.md`
`MD 3/.claude/skills/*` 4개	`.claude/skills/*` — 이 프로젝트에 맞게 다시 작성(아래 5장)
`MD 3/Part1.png` , `Part2.png` (검토 메모)	`CLAUDE.md` 스킬 호출 방식, `RECOMMENDED.md` 8장, `CONVENTIONS.md` .gitignore
(신규)	`GEMINI.md` , `.claude/settings.json` , `docs/guides/RECOMMENDED.md` , `docs/logs/`

충돌을 정리한 방식

항목	문서별 차이	통합본 처리
모델 서버 폴더명	`model-server` (JWC·M3) / `ml-server` (AV)	다수안 `model-server` 로 통일, 학습 코드는 AV안 `ml/` 유지. 최종은 D-14
직군 분류	GH·JWC 3분류 / AV 4분류(NEW_GRAD 포함)	10-30 팀 결정대로 3분류. DATA_AND_MODEL·DATABASE·API_SPEC 수정
등급	AV A~E 5등급 / features·decisions 3등급	Good·Standard·Poor 3단계, 경계는 설정값 `TODO(D-01)`
인증	features "세션" / AV·M3 JWT	10-30 팀 결정대로 토큰(JWT). features 원본 시트 갱신 필요 표시
관리자 역할	features 4역할 / AV 2역할	10-30 팀 결정대로 관리자 1개(`ROLE_ADMIN`)
커밋 메시지	README·JWC `feat - 내용` / AV·M3 `feat: 내용`	저장소 README 확정안 유지, D-17로 재논의 여지 기록
브랜치명	JWC `feature/기능명` / AV·M3 `feature/{티켓}-{설명}`	`feature/FP-12-…` 권장, 티켓 없으면 `feature/기능명` 허용
Jira 키	AV 예시 `AVN`	실제 Jira 프로젝트 키 `FP` 로 교체
응답 형식	AV `error{code,message}` / M3 `message, errorCode`	AV안(API_SPEC) 기준, D-20 기록
타임아웃	AV 1초/3초 / M3 3초/5초	설정값으로 분리, D-18 기록

항목	문서별 차이	통합본 처리
FastAPI 장애 시	M3 "간이 점수 반환 가능" / AV "점수는 항상 모델이 계산"	대체 점수 금지(AV·JWC 원칙 우선), 에러 코드로 응답
기능 번호	AV 기능 0·2·3·5 / features N-ID	API_SPEC·ARCHITECTURE 상단에 대응표 추가, N-ID가 기준
AI 규칙 원본 파일	AV `CLAUDE.md` 원본 / GH·JWC `AGENTS.md` 원본	`AGENTS.md` 원본(Claude·GPT·Gemini 공통), `CLAUDE.md` · `GEMINI.md` 는 불러오기만

2026-10-31에 이 통합본을 다시 AI용 md / 사람용 PDF / 공통 로그로 나눴습니다. 바뀐 내용은 `docs/logs/2026-10-31_문서담당_문서구조-정리.md` 를 보세요.

11 용어집

AI가 낯선 단어를 쓰면 길게 설명을 시키지 말고 여기서 먼저 찾아보세요. 한도를 아끼는 가장 쉬운 방법입니다.

AI · 바이브 코딩

바이브 코딩	코드를 직접 다 쓰는 대신 AI에게 말로 요청해서 코드를 만들고, 사람은 확인·수정하는 개발 방식.
프롬프트	AI에게 보내는 요청 문장. 4장의 템플릿이 좋은 프롬프트의 틀.
토큰 (AI)	AI가 글을 처리하는 단위. 대화가 길고 붙여 넣는 양이 많을수록 많이 쓰고, 무료 한도가 빨리 찬다. 로그인용 "토큰(JWT)"과는 다른 말.
컨텍스트	AI가 한 대화에서 기억하고 있는 내용 전체. 너무 길어지면 앞 내용을 헷갈린다 → 새 대화로.
md (마크다운)	`#`, `-` 같은 기호로 제목·목록을 표시하는 텍스트 파일 형식. 확장자 `.md`.
AGENTS.md / CLAUDE.md / GEMINI.md	코딩 도구가 대화를 시작할 때 자동으로 읽는 규칙 파일. 원본은 AGENTS.md.
Claude Code	Claude가 내 프로젝트 폴더의 파일을 직접 읽고 고치는 코딩 도구. VS Code 확장으로 쓸 수 있다.
plan 모드 (계획 모드)	AI가 코드를 바로 고치지 않고 계획만 먼저 보여 주는 모드. 여러 파일을 바꾸는 작업 전에 쓴다.
확인 창 (권한 요청)	AI가 CSS 수정· `git push` 같은 작업을 하기 전에 허락을 묻는 창. 이유를 읽고 누른다.
스킬	Claude Code에서 특정 작업용 매뉴얼을 넣어 두는 기능. 지금은 쓰지 않는다 (3장).
웹 채팅	브라우저·앱에서 쓰는 Claude·ChatGPT·Gemini 대화창. 저장소 파일을 못 읽어서 PASTE.md를 붙여 넣는다.

우리 문서

N-ID (N-12 등)	기능 명세서의 기능 번호. 요청·커밋·주석에 항상 붙인다. 목록은 `docs/ai/features.md`.
D-xx (D-01 등)	`decisions.md`의 결정 사항 번호. 미정 항목은 AI가 정하지 않고 팀이 정한다.
P0 ~ P4	기능 우선순위. P0 필수 · P1 핵심 흐름 · P2 여유 있을 때 · P3 제외 권장 · P4 정의 미정.
작업 로그	DB·API 등 여러 곳을 바꾼 작업의 기록. `docs/logs/`에 작업마다 파일 하나.
검수	로그·작업물을 지정된 담당이 확인하는 것.
TODO(D-xx)	"이 값은 D-xx가 정해지면 바꿔야 함"이라는 코드 주석 표시.

Git · 협업

저장소 (repo)	프로젝트 코드 전체와 변경 기록이 들어 있는 곳. 우리는 GitHub를 쓴다.
브랜치	각자 작업하는 갈래. `main`(발표용 안정본) · `develop`(합치는 곳) · `feature/…`(개인 작업).
커밋 (commit)	변경 내용을 한 단위로 저장하는 것. 메시지 형식 `행위 - 내용`.
pull / push	pull: 서버의 최신 코드를 받아 오기. push: 내 커밋을 서버에 올리기.
PR (Pull Request)	"내 브랜치를 develop에 합쳐 주세요"라는 요청. 검토를 거쳐 병합된다.
병합 (merge)	두 브랜치의 변경을 하나로 합치는 것.

충돌 (conflict)	두 사람이 같은 곳을 다르게 바꿔서 자동으로 합칠 수 없는 상태. 남의 코드를 지우지 말고 같이 본다.
diff	무엇이 바뀌었는지 보여 주는 비교 화면. 커밋 전에 꼭 본다.
Jira (FP)	팀 작업(이슈) 관리 도구. 브랜치 이름에 `FP-12` 같은 이슈 키를 넣으면 연결된다.

개발

프론트엔드 / 백엔드	프론트: 사용자가 보는 화면(React). 백엔드: 로그인·DB·계산을 처리하는 서버(Spring Boot).
모델 서버	신용점수와 SHAP을 계산하는 별도 서버(FastAPI). 백엔드만 호출한다.
API / 엔드포인트	API: 서버끼리 주고받는 약속. 엔드포인트: 그 약속의 주소(예: `/api/v1/...`).
요청 · 응답 필드	API로 보내고 받는 데이터의 항목 이름. 바꾸면 다른 기능이 깨질 수 있어 로그 대상.
DTO	API로 주고받는 데이터를 담는 Java 클래스.
DB · 테이블 · 컬럼	데이터 저장소(Oracle) · 엑셀의 시트 같은 표 · 표의 열.
DDL	테이블을 만들거나 바꾸는 SQL 문(`CREATE TABLE` 등).
스키마	데이터의 구조(어떤 항목이 어떤 형식으로 들어가는지)를 정의한 것.
mock (목)	상대 서버가 아직 없을 때 쓰는 가짜 응답. 실제 응답과 같은 모양으로 만든다.
JWT (토큰 인증)	로그인하면 서버가 발급하는 출입증. 요청마다 함께 보내 본인임을 증명한다.
SHAP	모델 점수에 각 항목이 얼마나 영향을 줬는지 보여 주는 방법. 원인(인과)을 증명하지는 않는다.
합성 데이터	실제 사람 정보가 아니라 규칙에 따라 만들어 낸 가짜 데이터.
마스킹	개인정보 일부를 가리는 것(예: 010-****-1234).
하드코딩 / 상수·설정값	하드코딩: 값을 코드 곳곳에 직접 박아 넣는 것. 상수·설정값: 한곳에 모아 이름을 붙여 두는 것 → 바꾸기 쉽다.
의존성 / 라이브러리	프로젝트가 가져다 쓰는 외부 코드. `pom.xml` · `package.json`에 적힌다.
환경 변수 / .env	비밀번호·API 키처럼 코드에 직접 쓰면 안 되는 값을 따로 두는 곳. 절대 올리지 않는다.
Swagger	백엔드 API를 브라우저에서 직접 호출해 볼 수 있는 화면.
리팩터링	동작은 그대로 두고 코드 구조만 다듬는 것. 요청 없이 시키지 않는다.